



BÀI THU HOẠCH: TÌM HIỂU VỀ THƯ VIỆN REGULAR EXPRESSION

Regular Expression

Đây là một công cụ hữu ích trong lập trình, giúp tìm kiếm, kiểm tra và xử lý chuỗi một cách hiệu quả. Bài thu hoạch này sẽ trình bày tổng quan về Regex, cú pháp cơ bản, ứng dụng thực tế và cách sử dụng thư viện Regex trong Python để xử lý dữ liệu. Qua đó, giúp người đọc hiểu rõ hơn về cách áp dụng Regex trong lập trình và phân tích dữ liệu.

Chu Quang Cường - 24520236
cuongchu200@gmail.com

MỤC LỤC

1. Giới thiệu về Regular Expression:.....	2
2. Cách sử dụng Regex trong Python:	2
2.1. Hàm trong Regex	2
2.1.1. Hàm re.findall()	3
2.1.2. Hàm re.compile()	3
2.1.3. Hàm re.split()	3
2.1.4. Hàm re.sub()	4
2.1.5. Hàm re.subn()	5
2.1.6. Hàm re.escape()	5
2.1.7. Hàm re.search().....	6
2.2. Meta-characters	7
2.2.1. \ - Dấu gạch chéo ngược	8
2.2.2. [] – Dấu ngoặc vuông	8
2.2.3. ^ - Dấu mũ	9
2.2.4. \$ - Dollar	10
2.2.5. . – Dấu chấm	10
2.2.6. - Hoặc	11
2.2.7. ? – Dấu chấm hỏi	11
2.2.8. * - Ngôi sao	11
2.2.9 + - Cộng	11
2.2.10. {m, n} – Dấu ngoặc nhọn	12
2.2.11. (<regex>) – Nhóm	12
2.3. Chuỗi đặc biệt	13
2.4. Bộ ghép ký tự	14
3. Đối tượng phù hợp.....	15
3.1. Lấy chuỗi và biểu thức chính quy	15
3.2. Lấy chỉ số của đối tượng khớp	15
3.3. Nhận chuỗi con phù hợp	16

1. Giới thiệu về Regular Expression:

Regular Expression hay **RegEx** là một chuỗi ký tự đặc biệt sử dụng mẫu tìm kiếm để tìm một chuỗi hoặc một tập hợp các chuỗi. Nó có thể phát hiện sự có mặt hoặc vắng mặt của một văn bản bằng cách so sánh nó với một mẫu cụ thể và cũng có thể chia một mẫu thành một hoặc nhiều mẫu con.

- Regex Module in Python

Python có 1 module tích hợp có tên là “re” được sử dụng cho các biểu thức chính quy. Và chúng ta có thể gọi đến module này thông qua câu lệnh **import**.

```
# importing re module
import re
```

2. Cách sử dụng Regex trong Python:

2.1. Hàm trong Regex

- `re.findall()`: Tìm và trả về tất cả các lần xuất hiện khớp trong một danh sách.
- `re.compile()`: Biểu thức chính quy được biên dịch thành các đối tượng mẫu.
- `re.split()`: Phân chia chuỗi theo số lần xuất hiện của một ký tự hoặc một mẫu.
- `re.sub()`: Thay thế tất cả các lần xuất hiện của một ký tự hoặc mẫu bằng một chuỗi thay thế.
- `re.escape()`: Thoát khỏi ký tự đặc biệt.
- `re.search()`: Tìm kiếm lần xuất hiện đầu tiên của ký tự hoặc mẫu

Chi tiết hơn, ta sẽ có định nghĩa và những ví dụ sau đây:

2.1.1. Hàm re.findall()

Trả về tất cả các kết quả khớp không trùng nhau của mẫu trong chuỗi, dưới dạng danh sách các chuỗi. Chuỗi được quét từ trái sang phải và các kết quả khớp được trả về theo thứ tự tìm thấy.

Ví dụ: Tìm tất cả các lần xuất hiện của một mẫu

Phần lập trình này sẽ dùng regex (**\d+**) để tìm tất cả các chuỗi của một hoặc nhiều chữ số trong chuỗi đã cho sau đó lưu trữ chúng vào 1 danh sách

```
import re
s = "Hello my spy code is 134568 and here is my enemy's code: 241234"
regex = '\d+'

matching = re.findall(regex, s)
print(matching)
# ['134568', '241234']
```

2.1.2. Hàm re.compile()

Regular Expression được biên dịch thành các đối tượng mẫu, có các phương thức thực hiện nhiều thao tác khác nhau như tìm kiếm mẫu phù hợp hoặc thực hiện thay thế chuỗi.

Ví dụ:

Code sử dụng mẫu regex để tìm và liệt kê tất cả các chữ cái thường từ 'a' đến 'h' trong chuỗi đầu vào.

```
import re
p = re.compile('[a-h]')

print(p.findall("Where there's a will, there's a way"))
# ['h', 'e', 'e', 'h', 'e', 'e', 'a', 'h', 'e', 'e', 'a', 'a']
```

2.1.3. Hàm re.split()

Chia chuỗi theo số lần xuất hiện của một ký tự hoặc một mẫu, khi tìm thấy mẫu đó, các ký tự còn lại trong chuỗi sẽ được trả về dưới dạng một phần của danh sách kết quả.

Cú pháp:

```
re.split(pattern, string, maxsplit=0, flags=0)
```

Tham số đầu tiên, **pattern** biểu thị regex, **string** là chuỗi đã cho mà **pattern** sẽ được tìm kiếm và chia tách, **maxsplit** nếu không được cho sẽ được coi là 0 và nếu

bất kỳ giá trị nào khác không được cho, thì nhiều nhất là số lần chia tách xảy ra. Nếu **maxsplit** = 1 thì chuỗi sẽ chỉ tách một lần, tạo ra danh sách có độ dài là 2. Các **flag** rất hữu ích và có thể giúp rút ngắn mã, chúng không phải là tham số cần thiết, ví dụ: **flags** = **re.IGNORECASE**, tức là chữ thường hoặc chữ hoa sẽ bị bỏ qua.

Ví dụ:

```
import re
print(re.split('\W+', 'On 28th Mar 2025, at 10:00 AM'))
print(re.split('\d+', 'On 28th Mar 2025, at 10:00 AM'))
print(re.split('\d+', 'On 28th Mar 2025, at 10:00 AM', 1))
print(re.split('[a-f]+', 'All work and no play makes Jack a dull boy',
flags=re.IGNORECASE))
print(re.split('[a-f]+', 'All work and no play makes Jack a dull boy'))
# ['On', '28th', 'Mar', '2025', 'at', '10', '00', 'AM']
# ['On ', 'th Mar ', ', at ', ':', ' AM']
# ['On ', 'th Mar 2025, at 10:00 AM']
# ['', 'll work ', 'n', ' no pl', 'y m', 'k', 's J', 'k ', ' ', 'ull ', 'oy']
# ['All work ', 'n', ' no pl', 'y m', 'k', 's J', 'k ', ' ', 'ull ', 'oy']
```

2.1.4. Hàm re.sub()

'Sub' trong hàm là viết tắt của **SubString**, một **pattern** regex nhất định được tìm kiếm trong chuỗi đã cho (tham số thứ 3) và khi tìm thấy mẫu chuỗi con được thay thế bằng **repl** (tham số thứ 2), kiểm tra **count** và duy trì số lần điều này xảy ra.

Cú pháp:

```
re.sub(pattern, repl, string, count=0, flags=0)
```

Ví dụ:

- Câu lệnh đầu tiên thay thế tất cả các lần xuất hiện của 'lo' bằng '~*' (không phân biệt chữ hoa chữ thường): . **'~*ve me, ~*ve my dog'**
- Câu lệnh thứ hai thay thế tất cả các lần xuất hiện của 'lo' bằng '~*' (phân biệt chữ hoa chữ thường): . **'Love me, ~*ve my dog'**
- Câu lệnh thứ ba thay thế lần xuất hiện đầu tiên của 'lo' bằng '~*' (không phân biệt chữ hoa chữ thường): . **'~*ve me, love my dog'**
- Thứ tư thay thế 'AND' bằng ' & ' (không phân biệt chữ hoa chữ thường): **'Set your target & keep trying until you reach it'**

```
import re
print(re.sub('lo', '~*', 'Love me, love my dog',
            flags=re.IGNORECASE))
print(re.sub('lo', '~*', 'Love me, love my dog'))
print(re.sub('lo', '~*', 'Love me, love my dog',
            count=1, flags=re.IGNORECASE))
print(re.sub(r'\sAND\s', ' & ', 'Set your target and keep trying until you reach
it',
            flags=re.IGNORECASE))
# ~*ve me, ~*ve my dog
# Love me, ~*ve my dog
# ~*ve me, love my dog
# Set your target & keep trying until you reach it
```

2.1.5. Hàm re.subn()

Hàm subn() tương tự sub() về mọi mặt, trừ cách cho đầu ra. Nó trả về một tuple với số lượng tổng số thay thế và chuỗi mới thay vì chỉ chuỗi.

Cú pháp:

```
re.subn(pattern, repl, string, count=0, flags=0)
```

Ví dụ:

Hàm re.subn() thay thế tất cả các lần xuất hiện của một mẫu trong một chuỗi và trả về một bộ với chuỗi đã sửa đổi và số lần thay thế được thực hiện. Nó hữu ích cho cả việc thay thế phân biệt chữ hoa chữ thường và ngược lại.

```
import re
print(re.subn('lo', '~*', 'Love me, love my dog'))
t = re.subn('lo', '~*', 'Love me, love my dog',
            flags=re.IGNORECASE)
print(t)
print(len(t))
print(t[0])
# ('Love me, ~*ve my dog', 1)
# (~*ve me, ~*ve my dog', 2)
# 2
# ~*ve me, ~*ve my dog
```

2.1.6. Hàm re.escape()

Trả về chuỗi với tất cả các ký tự không phải chữ và số được gạch chéo ngược, hữu ích khi muốn khớp với một chuỗi có thể có regex metacharacters trong đó.

Cú pháp:

```
re.escape(string)
```

Ví dụ:

Hàm `re.escape()` được sử dụng để escape các ký tự đặc biệt trong một chuỗi, giúp nó an toàn khi được sử dụng như một mẫu trong regex. Nó đảm bảo rằng bất kỳ ký tự nào có ý nghĩa đặc biệt trong regex đều được coi là ký tự theo nghĩa đen.

```
import re
print(re.escape("You open this file in 1 second"))
print(re.escape("That's really fast but what's this [f-9], just one \t thing"))
# You\ open\ this\ file\ in\ 1\ second
# That's\ really\ fast\ but\ what's\ this\ \[f-9\],\ just\ one\ \t\ thing
```

2.1.7. Hàm `re.search()`

Phương pháp này trả về `None` (nếu mẫu không khớp) hoặc `re.MatchObject` chứa thông tin về phần khớp của chuỗi. Phương pháp này dừng ngay sau lần khớp đầu tiên, do đó phù hợp nhất để kiểm tra 1 regex hơn là trích xuất dữ liệu.

Ví dụ: Tìm kiếm sự xuất hiện của mẫu

Đoạn code sử dụng regex để tìm kiếm một mẫu trong chuỗi đã cho. Nếu tìm thấy kết quả khớp, nó sẽ trích xuất và in các phần khớp của chuỗi.

Trong ví dụ cụ thể này, nó tìm kiếm một mẫu bao gồm một tháng (chữ cái) theo sau là một ngày (chữ số) trong chuỗi đầu vào “I was born on October 14”. Nếu tìm thấy kết quả khớp, nó sẽ in ra kết quả khớp đầy đủ, tháng và ngày.

```
import re
regex = r"([a-zA-Z]+) (\d+)"

match = re.search(regex, "I was born on October 14")
if match != None:
    print ("Match at index %s, %s" % (match.start(), match.end()))
    print ("Full match: %s" % (match.group(0)))
    print ("Month: %s" % (match.group(1)))
    print ("Day: %s" % (match.group(2)))
else:
    print ("The regex pattern does not match.")

# Match at index 14, 24
# Full match: October 14
# Month: October
# Day: 14
```

2.2. Meta-characters

Metacharacters là những ký tự có ý nghĩa đặc biệt.

Để hiểu phép loại suy RE, Metacharacters rất hữu ích và quan trọng. Chúng được sử dụng trong các hàm của module re. Dưới đây là danh sách các metacharacters.

- \ Dùng để bỏ đi ý nghĩa đặc biệt của ký tự theo sau nó.
- [] Biểu diễn một lớp ký tự.
- ^ Khớp với phần đầu.
- \$ Khớp với phần cuối.
- . Khớp với bất kỳ ký tự nào ngoại trừ ký tự xuống dòng.
- | Nghĩa là HOẶC (Khớp với bất kỳ ký tự nào được phân tách bằng nó).
- ? Khớp với 0 hoặc 1 lần xuất hiện.
- * Bất kỳ số lần xuất hiện nào (bao gồm 0 lần xuất hiện).
- + Một hoặc nhiều lần xuất hiện.
- {} Chỉ ra số lần xuất hiện của regex trước đó để khớp.
- () Bao gồm một nhóm Regex.

Hãy nói chi tiết hơn về chúng sau đây:

2.2.1. \ - Dấu gạch chéo ngược

Dấu gạch chéo ngược (\) đảm bảo rằng ký tự không được xử lý theo cách đặc biệt. Vì vậy có thể được coi là một cách escape metacharacters.

Ví dụ, nếu bạn muốn tìm kiếm dấu chấm(.) trong chuỗi thì bạn sẽ thấy dấu chấm(.) sẽ được coi là một ký tự đặc biệt vì là một trong các metacharacters (như được hiển thị trong bảng trên). Vì vậy, đối với trường hợp này, chúng ta sẽ sử dụng dấu gạch chéo ngược(\) ngay trước dấu chấm(.) để nó mất đi tính đặc biệt của nó.

Ví dụ:

Dù tìm kiếm đầu tiên **re.search(r'.', s)** khớp với bất kỳ ký tự nào, nhưng tìm kiếm thứ hai **re.search(r'\.', s)** tìm kiếm và khớp cụ thể với ký tự dấu chấm.

```
import re
s = 'r1muru.n0.pr0'

# without using \
match = re.search(r'.', s)
print(match)

# using \
match = re.search(r'\.', s)
print(match)
# <re.Match object; span=(0, 1), match='r'>
# <re.Match object; span=(6, 7), match='.'>
```

2.2.2. [] – Dấu ngoặc vuông

Dấu ngoặc vuông biểu diễn một lớp ký tự bao gồm một tập hợp các ký tự mà chúng ta muốn khớp. Chúng ta có thể chỉ định một phạm vi ký tự (-). Ví dụ,

- [0-7] là mẫu như [01234567]
- [a-f] giống như [abcdef]

Chúng ta cũng có thể đảo ngược lớp ký tự bằng cách sử dụng ký hiệu dấu mũ(^). Ví dụ,

- [^0-3] nghĩa là bất kỳ ký tự nào ngoại trừ 0, 1, 2 hoặc 3
- [^a-c] nghĩa là bất kỳ ký tự nào ngoại trừ a, b hoặc c

Ví dụ:

Trong đoạn code này, bạn đang sử dụng regex để tìm tất cả các ký tự trong chuỗi nằm trong phạm vi từ 'a' đến 'i' và từ '0' đến '5'. Hàm **re.findall()** trả về danh sách tất cả các ký tự như vậy.

```
import re

s = "und3r57AnD1n9_r393x_15_51mP13_a5_7Ha7"
pattern = "[a-i0-5]"
res = re.findall(pattern, s)

print(res)
# ['d', '3', '5', '1', '3', '3', '1', '5', '5', '1', '3', 'a', '5', 'a']
```

2.2.3. ^ - Dấu mũ

Dấu mũ khớp với phần đầu của chuỗi nghĩa là kiểm tra xem chuỗi có bắt đầu bằng ký tự đã cho hay không. Ví dụ:

- `^u` sẽ kiểm tra xem chuỗi có bắt đầu bằng u hay không, chẳng hạn như under, up, unable, v.v.
- `^in` sẽ kiểm tra xem chuỗi có bắt đầu bằng in không, chẳng hạn như index, information, v.v.

Ví dụ:

Nếu chuỗi bắt đầu bằng **"Di"**, chuỗi đó sẽ được đánh dấu là **"Matched"**, nếu không, chuỗi đó sẽ được gắn nhãn là **"Not matches"**.

```
import re
regex = r'^Di'
lst = ['Diligence is the mother of success', 'The die is cast', 'Diamond cuts diamond']
for s in lst:
    if re.match(regex, s):
        print(f'Matched: {s}')
    else:
        print(f'Not matched: {s}')
# Matched: Diligence is the mother of success
# Not matched: The die is cast
# Matched: Diamond cuts diamond
```

2.2.4. \$ - Dollar

Biểu tượng Dollar khớp với phần cuối của chuỗi tức là kiểm tra xem chuỗi có kết thúc bằng ký tự đã cho hay không. Ví dụ:

- `r$` sẽ kiểm tra chuỗi kết thúc bằng `a` như `dominator`, `dollar`, v.v.
- `er$` sẽ kiểm tra chuỗi kết thúc bằng `er` như `former`, `ladder`, `leader`, v.v.

Ví dụ:

Đoạn code này sử dụng regex để kiểm tra xem chuỗi có kết thúc bằng “**wisdom**”, Nếu khớp, nó sẽ in ra “**Matched**”, ngược lại nó sẽ in ra “**Not matched**” .

```
import re

s = "With age comes wisdom"
pattern = r"wisdom"

match = re.search(pattern, s)
if match:
    print("Matched")
else:
    print("Not matched")
# Matched
```

2.2.5. . – Dấu chấm

Biểu tượng Dot(.) chỉ khớp với một ký tự duy nhất ngoại trừ ký tự xuống dòng (`\n`). Ví dụ –

- `ab` sẽ kiểm tra chuỗi có chứa bất kỳ ký tự nào tại vị trí dấu chấm như `acb`, `acbd`, `abbb`, v.v.
- `..` sẽ kiểm tra xem chuỗi có chứa ít nhất 2 ký tự không

Ví dụ:

Dấu chấm trong “**a.s and**” biểu diễn bất kỳ ký tự nào. Nếu khớp, nó sẽ in ra “**Matched**”, ngược lại nó sẽ in ra “**Not matched**”.

```
import re

s = "Raining cats and dogs"
pattern = r"a.s and"

match = re.search(pattern, s)
if match:
    print("Matched")
else:
    print("Not matched")
# Matched
```

2.2.6. | - Hoặc

Ký hiệu | hoạt động như toán tử OR nghĩa là nó kiểm tra xem mẫu trước hoặc sau có trong chuỗi hay không. Ví dụ: a|c sẽ khớp với bất kỳ chuỗi nào chứa a hoặc c như animal, cat, cow, v.v.

2.2.7. ? – Dấu chấm hỏi

Dấu chấm hỏi là một lượng từ trong regex cho biết phần tử trước đó phải được khớp 0 hoặc 1 lần. Nó cho phép bạn chỉ định rằng phần tử là tùy chọn, nghĩa là nó có thể xuất hiện một lần hoặc không xuất hiện lần nào. Ví dụ: ab?c sẽ khớp với chuỗi ac, acb, dabc nhưng sẽ không khớp với abbc vì có hai b. Tương tự, nó sẽ không khớp với abdc vì b không theo sau c.

2.2.8. * - Ngôi sao

Biểu tượng sao khớp với không hoặc nhiều lần xuất hiện của regex trước biểu tượng *. Ví dụ : ab*c sẽ khớp với chuỗi ac, abc, abbbc, dabc, v.v. nhưng sẽ không khớp với abdc vì b không theo sau c.

2.2.9 + - Cộng

Biểu tượng cộng (+) khớp với một hoặc nhiều lần xuất hiện của regex trước biểu tượng +. Ví dụ: ab+c sẽ khớp với chuỗi abc, abbc, dabc, nhưng sẽ không khớp với ac, abdc, vì không có b trong ac và b, không theo sau là c trong abdc.

2.2.10. {m, n} – Dấu ngoặc nhọn

Dấu ngoặc nhọn khớp với bất kỳ sự lặp lại nào trước regex từ m đến n bao gồm cả hai. Ví dụ: $a\{2, 4\}$ sẽ khớp với chuỗi aaab, baaaac, gaad, nhưng sẽ không khớp với các chuỗi như abc, bc vì chỉ có một a hoặc không có a trong cả hai trường hợp.

2.2.11. (<regex>) – Nhóm

Biểu tượng nhóm được sử dụng để nhóm các mẫu phụ. Ví dụ: $(a|b)cd$ sẽ khớp với các chuỗi như acd, abcd, gacd, v.v.

2.3. Chuỗi đặc biệt

Trình tự đặc biệt không khớp với ký tự thực tế trong chuỗi mà thay vào đó, nó cho biết vị trí cụ thể trong chuỗi tìm kiếm nơi khớp phải xảy ra. Nó giúp viết các mẫu thường dùng dễ dàng hơn. Sau đây là danh sách các chuỗi đặc biệt:

- `\A` Phù hợp nếu chuỗi bắt đầu bằng ký tự đã cho.

E.x: `\Ain` in the world, in Vietnam

- `\b` Phù hợp nếu từ bắt đầu hoặc kết thúc bằng ký tự đã cho. `\b(chuỗi)` sẽ kiểm tra phần đầu của từ và `(chuỗi)\b` sẽ kiểm tra phần cuối của từ.

E.x: `\bthi` think, this

E.x: `rd\b` bird, third

- `\B` Trái với `\b`, chuỗi không bắt đầu hoặc kết thúc bằng regex đã cho.

E.x: `\Bthi` doll, lamp

E.x: `rd\b` apple, ball

- `\d` Phù hợp với bất kỳ chữ số thập phân nào, tương đương với lớp `[0-9]`

E.x: `\d` 1410, r1muru

- `\D` Phù hợp với bất kỳ ký tự nào không phải chữ số, nghĩa là lớp `[^0-9]`

E.x: `\D` iaio, rimuru

- `\s` Phù hợp với bất kỳ ký tự khoảng trắng nào.

E.x: `\s` i ai o, ri mu ru

- `\S` Phù hợp với bất kỳ ký tự không phải khoảng trắng nào.

E.x: `\S` shirt, meat

- `\w` Phù hợp với bất kỳ ký tự chữ và số nào, nghĩa là lớp `[a-zA-Z0-9_]`

E.x: `\w` October14th, b33f

- `\W` Phù hợp với bất kỳ ký tự nào không phải chữ và số.

E.x: `\W` >\$!, &`<

- `\Z` Phù hợp nếu chuỗi kết thúc bằng regex đã cho.

E.x: `ll\Z` chall, roll

2.4. Bộ ghép ký tự

Set là một tập hợp các ký tự được bao trong dấu ngoặc '['']. **Set** được sử dụng để khớp với một ký tự duy nhất trong tập hợp các ký tự được chỉ định giữa các dấu ngoặc. Dưới đây là danh sách các **Set**:

- `\{n,\}` Định lượng ký tự hoặc nhóm trước và khớp với ít nhất n lần.
- `*` Định lượng ký tự hoặc nhóm trước và khớp 0 hoặc nhiều lần.
- `[ab12]` Phù hợp với các chữ số được chỉ định (0, 1, 2 hoặc 3).
- `[^k4t]` Phù hợp với bất kỳ ký tự nào TRỪ k, 4 và t.
- `[a-u]` Phù hợp với bất kỳ chữ cái thường nào giữa a và u.
- `[a-zA-Z]` Phù hợp với bất kỳ ký tự nào giữa a và z, chữ thường HOẶC chữ hoa.
- `[0-9]` Phù hợp với bất kỳ chữ số nào giữa 0 và 9.
- `[0-5][0-9]` Phù hợp với bất kỳ số có hai chữ số nào từ 00 đến 59.
- `\d` Phù hợp với bất kỳ chữ số nào (0-9).
- `\D` Phù hợp với bất kỳ ký tự nào không phải chữ số.
- `\w` Phù hợp với bất kỳ ký tự chữ và số nào (a-z, A-Z, 0-9 hoặc _).

3. Đối tượng phù hợp

Đối tượng **Match** chứa tất cả thông tin về tìm kiếm và kết quả và nếu không tìm thấy kết quả nào khớp thì **None** sẽ được trả về. Hãy cùng xem một số phương thức và thuộc tính thường dùng của đối tượng **Match**.

3.1. Lấy chuỗi và biểu thức chính quy

Thuộc tính **match.re** trả về regex được truyền và thuộc tính **match.string** trả về chuỗi được truyền.

Ví dụ: Lấy chuỗi và regex của đối tượng khớp

Đoạn code này tìm kiếm chữ cái “**R**” tại các từ trong chuỗi “**Hi R1muru, how can I help you**” và in ra mẫu regex (**res.re**) và chuỗi gốc (**res.string**)

```
import re
s = "Hi R1muru, how can I help you?"
res = re.search(r"\bR", s)

print(res.re)
print(res.string)
# re.compile('\bR')
# Hi R1muru, how can I help you
```

3.2. Lấy chỉ số của đối tượng khớp

- **start()** trả về chỉ số bắt đầu của chuỗi con khớp.
- **end()** trả về chỉ mục kết thúc của chuỗi con khớp.
- **span()** trả về một bộ chứa chỉ mục bắt đầu và kết thúc của chuỗi con khớp.

Ví dụ: Lấy chỉ mục của đối tượng khớp

Đoạn code tìm kiếm chuỗi con “**R1mu**” tại biên từ trong chuỗi “**Hi R1muru, how can I help you**” và in ra chỉ mục bắt đầu của kết quả khớp (**res.start()**), chỉ mục kết thúc của kết quả khớp (**res.end()**) và khoảng kết quả khớp (**res.span()**).

```
import re
s = "Hi R1muru, how can I help you?"
res = re.search(r"\bR1mu", s)

print(res.start()) # 3
print(res.end()) # 7
print(res.span()) # (3, 7)
```


3.3. Nhận chuỗi con phù hợp

Phương thức **group()** trả về phần chuỗi mà các mẫu khớp với nhau.

Ví dụ: Nhận chuỗi con phù hợp

Đoạn code này tìm kiếm chuỗi hai ký tự không phải chữ số theo sau là một khoảng trắng và chữ cái 'h' trong chuỗi “Hi R1muru, how can I help you” và in ra văn bản khớp với ký tự đó bằng cách sử dụng **res.group()**

```
import re
s = "Hi R1muru, how can I help you"
res = re.search(r"\D{3} h", s)
print(res.group()) # ru, h
```

Trong ví dụ trên, mẫu của chúng tôi chỉ định chuỗi chứa 3 ký tự theo sau là một khoảng trắng và khoảng trắng đó theo sau là chữ h.